

Санкт-Петербургский государственный университет

Искусственный интеллект и наука о данных

Группа 23.Б16-мм

MLP-Mixer и ConvMixer

Гришина Ксения Анатольевна

Доклад по дисциплине
«Алгоритмы обработки изображений»

Преподаватель:
Гавриленко О. Р.

Оглавление

Введение	3
1. MLP Mixer	4
1.1. Описание	4
1.2. Архитектура	4
1.3. Пример	6
1.4. Экспериментальные результаты	6
2. ConvMixer	10
2.1. Описание	10
2.2. Архитектура	10
2.3. Пример	11
2.4. Экспериментальные результаты	12
Заключение	14
Список литературы	15

Введение

Долгое время в области обработки изображений де-факто стандартом было использование CNN. После стали популярными VisualTransformers (ViT) появившиеся в 2021 году, которые демонстрировали впечатляющие результаты во многих задачах, часто превосходя классические сверточные архитектуры на больших наборах данных. Однако возникает вопрос: действительно ли сложные механизмы — свёртки или self-attention — необходимы?

В работах:

1. MLP-Mixer: An all-MLP Architecture for Vision — Tolstikhin et al., 2021 [1]
2. Patches Are All You Need? — Trockman Kolter, 2022 [2]

показано, что можно строить конкурентоспособные модели, используя гораздо более простые архитектуры.

Цель доклада — разобрать:

- идею этих архитектур,
- их устройство,
- примеры работы,
- экспериментальные результаты из работ.

1. MLP-Mixer

1.1. Описание

Часть тех же исследователей из Google Brains, работающими над ViT позднее предложили архитектуру MLP-Mixer (или сокращенно «Mixer»): конкурентоспособную, но концептуально и технически более простую альтернативу, которая не использует ни свертки, ни самовнимание. Вместо этого архитектура Mixer полностью основана на многослойных перцептронах (MLP), которые многократно применяются либо к пространственным координатам, либо к каналам признаков, используя только базовые процедуры умножения матриц, изменения формы, транспозиции и нелинейности.

Основная идея:

- изображение разбивается на небольшие патчи (как в ViT),
- затем применяется чередование двух типов MLP:
 - по каналам (по признакам),
 - по пространству (между патчами).

Таким образом пространственная информация и признаки обрабатываются раздельно, а модель остаётся очень простой.

В частном случае, архитектуру Mixer можно рассматривать как сверточную нейронную сеть (CNN), которая использует свертки 1×1 для смешивания каналов и одноканальные послойные свертки полного рецептивного поля и параметров для смешивания токенов. Однако обратное утверждение неверно, поскольку типичные CNN не являются частными случаями Mixer, так как свертка сложнее чем операции над матрицами.

1.2. Архитектура

Рассмотрим архитектуру (Рис. 1) подробнее. Модель принимает на вход изображение размера $H \times W \times 3$, которое сначала разбивается на неперекрывающиеся патчи фиксированного размера $P \times P$. Тогда число патчей: $S = \frac{HW}{P^2}$. Каждый такой патч разворачивается в вектор размерности $P^2 \cdot 3$ и проецируется в пространство размерности C : $x_i \in \mathbb{R}^{P^2 \cdot 3} \rightarrow z_i \in \mathbb{R}^C$, в результате чего получается набор векторов одинаковой размерности. Таким образом, изображение представляется в виде матрицы $X \in \mathbb{R}^{S \times C}$, где S — число патчей, а C — размер скрытого пространства.

Главная идея архитектуры состоит в том, что обработка изображения разбивается на два принципиально разных типа операций: обработку по пространству и обработку по каналам. В классических CNN эти операции переплетены внутри свёрток, а в Transformer — внутри attention. В MLP-Mixer они строго разделены.

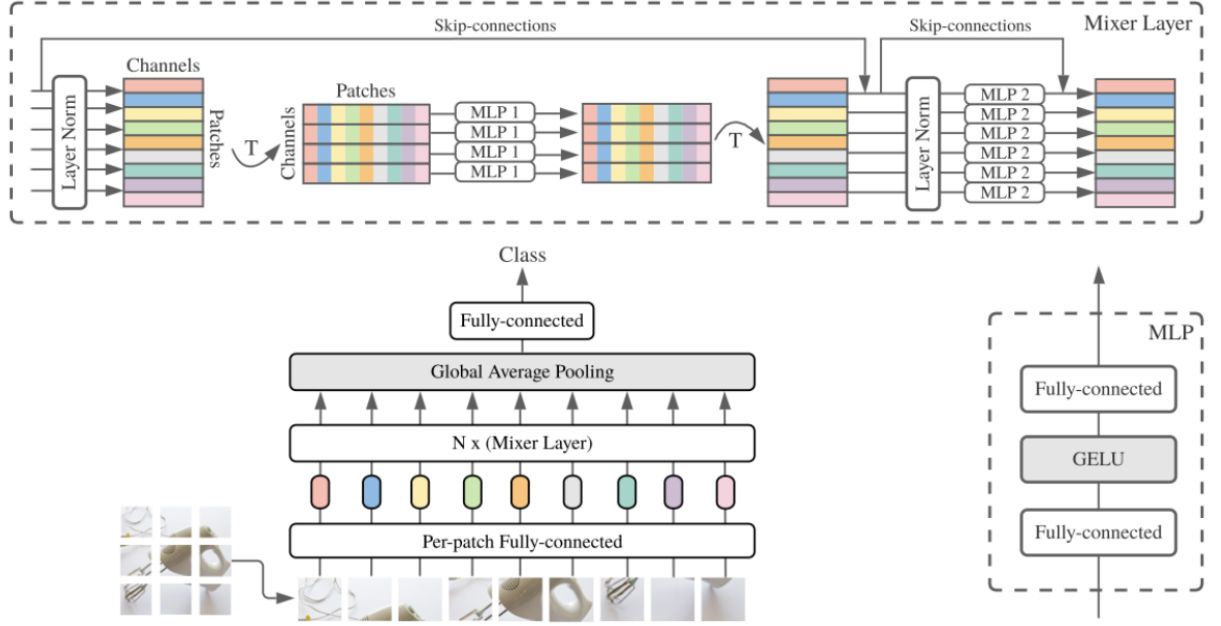


Рис. 1: Архитектура MLP Mixer

Каждый слой (Mixer layer) состоит из двух блоков.

Первый блок — это **token-mixing MLP**. Он отвечает за взаимодействие между разными патчами изображения, то есть за пространственную обработку. При этом он работает следующим образом: берётся один канал (один столбец матрицы), и рассматриваются значения этого канала во всех патчах. Таким образом, модель обрабатывает сразу всю картинку, но отдельно для каждого признака. Важно, что один и тот же MLP применяется ко всем каналам, что обеспечивает параметрическую эффективность и некоторую аналогию с инвариантностью свёрток. На этом этапе обрабатываются зависимости между патчами. Для каждого канала рассматривается вектор: $X_{:,j} \in \mathbb{R}^S$.

Далее применяется MLP: $\text{TokenMix}(X) = X + W_2 \sigma(W_1 \cdot \text{LayerNorm}(X)^T)^T$, где $W_1 \in \mathbb{R}^{S \times D}$, $W_2 \in \mathbb{R}^{D \times S}$, σ — функция активации (GELU). После этого применяется второй блок — **channel-mixing MLP**. Он работает в противоположном направлении: фиксируется один патч, и обрабатываются все его каналы. Это позволяет смешивать признаки внутри одного локального региона изображения. По сути, именно это и есть аналог операции 1×1 свёртки в CNN, но реализованный через полносвязную сеть.

Теперь фиксируется патч: $X_{i,:} \in \mathbb{R}^C$.

Применяется MLP: $\text{ChannelMix}(U) = U + W_4 \sigma(W_3 \cdot \text{LayerNorm}(U))$, где $W_3 \in \mathbb{R}^{C \times D'}$, $W_4 \in \mathbb{R}^{D' \times C}$.

Оба блока состоят из двух полносвязных слоёв с нелинейностью GELU между ними. Важную роль играют residual-связи и LayerNorm: каждый блок добавляется к входу, что стабилизирует обучение и позволяет строить глубокие модели.

Интересной особенностью является то, что архитектура не изменяет размер представления по глубине — она «изотропная». В отличие от CNN, где пространственное разрешение уменьшается, здесь размер матрицы остаётся постоянным. Это сближает MLP-Mixer с Transformer-подобными моделями.

Также важно, что token-mixing MLP имеет полный receptive field — он сразу видит все патчи. Это делает модель способной учитывать глобальный контекст уже на каждом слое, в отличие от CNN, где receptive field растёт постепенно.

Один полный слой имеет вид: $X \rightarrow X + \text{TokenMix}(X) \rightarrow X + \text{ChannelMix}(X)$.

Размер представления $S \times C$ остаётся неизменным на всех слоях.

Отдельно подчёркивается, что вычислительная сложность модели линейна по числу патчей, в отличие от Transformer, где она квадратичная. Это делает модель потенциально более масштабируемой.

1.3. Пример

Для изображения 224×224 и размера патча 16×16 : $S = \left(\frac{224}{16}\right)^2 = 196$. Таким образом, модель работает с матрицей $196 \times C$. По сути - с таблицей.

На Рис. 2 представлена реализация модели с использованием библиотеки pytorch.

1.4. Экспериментальные результаты

Теперь перейдём к экспериментальной части статьи. Авторы оценивают модель по трём основным критериям: точность классификации (Top-1 Ассигасу — это доля объектов, для которых $\arg \max_k p_k = y_{true}$), вычислительная стоимость обучения и скорость инференса. Основной акцент делается на том, насколько простая MLP-архитектура может конкурировать с более сложными моделями.

Эксперименты проводились на нескольких задачах:

- ILSVRC2012 “ImageNet” (1,3 млн обучающих примеров, 1 тыс. классов) и очищенные метки ReaL,
- CIFAR-10/100 (50 тыс. примеров, 10/100 классов),
- Oxford-IIIT Pets (3,7 тыс. примеров, 36 классов) и Oxford Flowers-102 (2 тыс. примеров, 102 класса),
- Visual Task Adaptation Benchmark (VTAB-1k), который состоит из 19 различных наборов данных, каждый с 1 тыс. обучающих примеров.

А обучалась модель на:

- ImageNet-1k ≈ 1.3 млн изображений, 1000 классов;

```

from torch import nn
import torch.nn.functional as F

# MLP: Linear -> GELU -> Dropout -> Linear
class MLP(nn.Module):
    def __init__(self, dim, exp=2, drop=0.):
        super().__init__()
        self.fc1 = nn.Linear(dim, dim * exp)
        self.fc2 = nn.Linear(dim * exp, dim)
        self.drop = nn.Dropout(drop)
    def forward(self, x):
        return self.drop(self.fc2(self.drop(F.gelu(self.fc1(x)))))

# MixerLayer = TokenMixing + ChannelMixing
class MixerLayer(nn.Module):
    def __init__(self, n_patches, dim, exp=2, drop=0.):
        super().__init__()
        self.norm1, self.norm2 = nn.LayerNorm(dim), nn.LayerNorm(dim)
        self.token_mlp = MLP(n_patches, exp, drop) # mixing по патчам
        self.channel_mlp = MLP(dim, exp, drop) # mixing по каналам
    def forward(self, x): # x: [B, N, D]
        x = x + self.token_mlp(self.norm1(x).transpose(1, 2)).transpose(1, 2)
        x = x + self.channel_mlp(self.norm2(x))
        return x

# MLP Mixer: patching -> Mixer layers -> pooling -> classifier
class MLP Mixer(nn.Module):
    def __init__(self, img_size=256, patch=16, in_ch=3,
                 dim=128, depth=8, exp=2, n_classes=10, drop=0.):
        super().__init__()
        n_patches = (img_size // patch) ** 2
        # patch embedding через conv
        self.patcher = nn.Conv2d(in_ch, dim, patch, patch)
        # стек Mixer слоёв
        self.mixer = nn.Sequential(*[
            MixerLayer(n_patches, dim, exp, drop)
            for _ in range(depth)
        ])
        self.head = nn.Linear(dim, n_classes)
    def forward(self, x):
        x = self.patcher(x) # [B, D, H', W']
        x = x.flatten(2).transpose(1, 2) # [B, N, D]
        x = self.mixer(x)
        return self.head(x.mean(1)) # pooling + классификация

```

Рис. 2: Реализация MLP Mixer (by jaketae)

- ImageNet-21k ≈ 14 млн изображений;
- JFT-300M ≈ 300 млн изображений.

Ключевой вывод состоит в том, что эффективность MLP-Mixer критически зависит от объёма данных. При обучении на очень большом датасете JFT-300M модель достигает точности порядка 87.9% (Рис. 3), что сопоставимо с лучшими CNN и ViT, однако при обучении на более небольших датасетах модель начинает уступать. Это связано с тем, что в ней отсутствуют индуктивные предположения, характерные для CNN, такие как локальность и сдвиговая инвариантность. В результате модели сложнее обучаться на ограниченных данных.

В статье отдельно исследуется влияние масштаба модели и данных (Рис. 4). Увеличение размера модели (глубины и ширины) улучшает качество, но эффект становится особенно заметным только при наличии большого объёма данных. Это

	ImNet top-1	ReaL top-1	Avg 5 top-1	VTAB-1k 19 tasks	Throughput img/sec/core	TPUv3 core-days
Pre-trained on ImageNet-21k (public)						
● HaloNet [51]	85.8	—	—	—	120	0.10k
● Mixer-L/16	84.15	87.86	93.91	74.95	105	0.41k
● ViT-L/16 [14]	85.30	88.62	94.39	72.72	32	0.18k
● BiT-R152x4 [22]	85.39	—	94.04	70.64	26	0.94k
Pre-trained on JFT-300M (proprietary)						
● NFNet-F4+ [7]	89.2	—	—	—	46	1.86k
● Mixer-H/14	87.94	90.18	95.71	75.33	40	1.01k
● BiT-R152x4 [22]	87.54	90.54	95.33	76.29	26	9.90k
● ViT-H/14 [14]	88.55	90.72	95.97	77.63	15	2.30k
Pre-trained on unlabelled or weakly labelled data (proprietary)						
● MPL [35]	90.0	91.12	—	—	—	20.48k
● ALIGN [21]	88.64	—	—	79.99	15	14.82k

Рис. 3: Сравнение моделей. Столбцы «ImNet» и «ReaL» относятся к исходной проверке ImageNet и очищенной версии ReaL. «Avg 5» обозначает среднюю производительность по всем пяти последующим задачам (ImageNet, CIFAR-10, CIFAR-100, Pets, Flowers).

типичное поведение для архитектур с низким inductive bias.

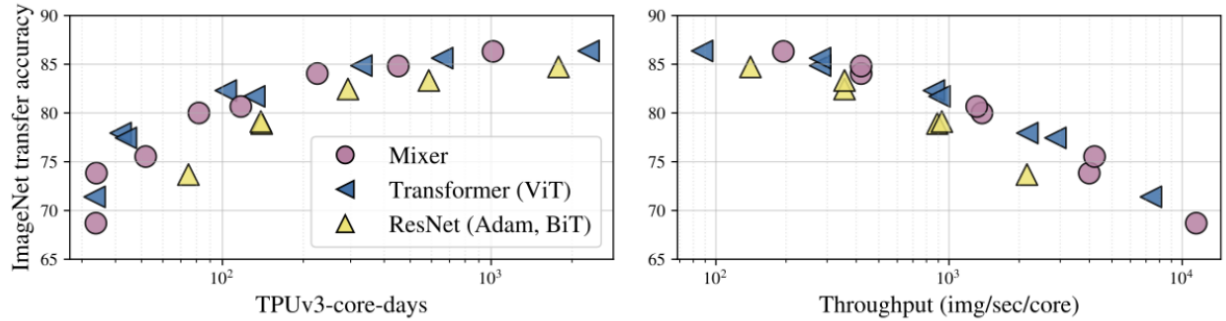


Рис. 4: Роль масштаба модели. Точность Top-1 валидации ImageNet в зависимости от общего объёма вычислений (слева) и пропускной способности (справа) моделей ViT, BiT и Mixer при различных масштабах. Все модели предварительно обучены на JFT-300M и дообучены при разрешении 224.

Ещё проводился эксперимент на инвариантность к входным перестановкам (Рис. 5). То есть проверка на то, как модели реагируют на перемешивание входных данных (пикселей и патчей изображения).

Брали две модели — Mixer-B/16 и ResNet50, обученных на JFT-300M, но с искажёнными входами следующим образом: перемешивают порядок патчей 16 × 16 и пиксели внутри них (одинаково для всех изображений); глобально полностью случайно переставляют все пиксели изображения.

Затем оценивали качество (5-shot accuracy на ImageNet). Авторы обнаружили, что Mixer почти не зависит от порядка патчей и пикселей внутри них — качество не меняется, тогда как ResNet сильно зависит от структуры изображения: при перестановке патчей качество резко падает. То есть Mixer менее чувствителен к пространственной структуре данных, а ResNet сильно опирается на локальные связи и порядок пикселей (сильный inductive bias CNN).

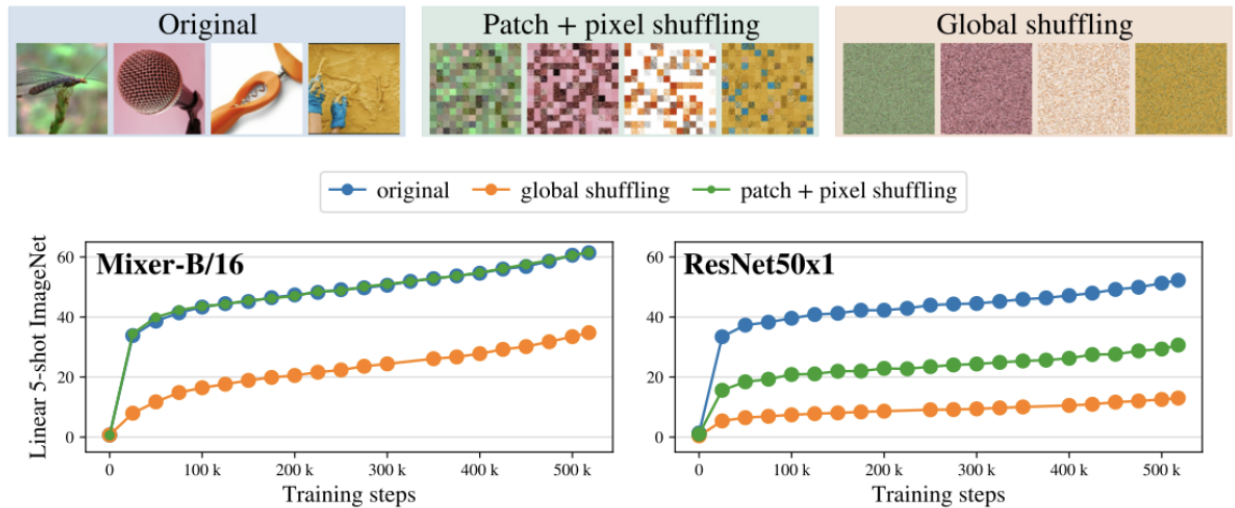


Рис. 5: Примеры входных данных из ImageNet до перестановки содержимого (слева); после перемешивания 16×16 фрагментов и пикселей внутри фрагментов (в центре); после глобального перемешивания пикселей (справа). Внизу: Mixer-B/16 (слева) и ResNet50x1 (справа), обученные с использованием трёх пайплайнов входных данных.

Главный вывод экспериментов заключается в том, что даже такая простая архитектура, как набор MLP, может быть конкурентоспособной — но только при наличии достаточного количества данных.

2. ConvMixer

2.1. Описание

В статье “Patches Are All You Need? ☒” 2022 года исследователей из Carnegie Mellon University и Bosch (спасибо за стиральные машины) пытаются упростить и переосмыслить успех Vision Transformer и MLP-Mixer, задались вопросом: действительно ли нужны сложные механизмы вроде attention, или ключевую роль играет сам факт разбиения изображения на патчи? Авторы ConvMixer показывают, что можно сохранить идею патчей, но заменить MLP на обычные свёртки, сохраняя идею раздельного смешивания каналов и пространства.

Авторы ConvMixer показывают, что можно сохранить идею патчей, но заменить MLP на свёртки, сохранив разделение пространственного и канального смешивания.

2.2. Архитектура

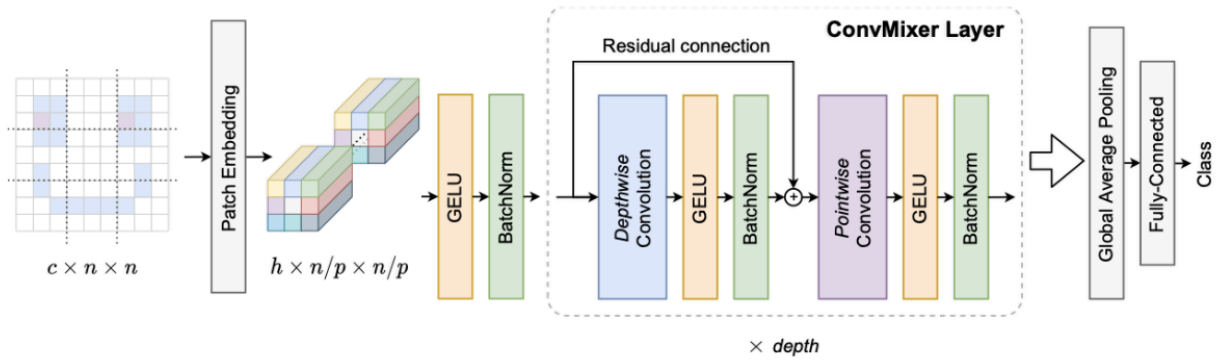


Рис. 6: Архитектура ConvMixer

Архитектура ConvMixer (Рис. 6) состоит из embedding слоя и повторяющихся блоков свёрток.

На первом этапе изображение преобразуется в набор патчей с помощью обычной свёртки. Используется свёртка с размером ядра, равным размеру патча, и шагом, равным этому же размеру:

$$z_0 = \text{BN}(\sigma(\text{Conv}_{c_{in} \rightarrow h}(X, \text{stride} = p, \text{kernel_size} = p)))$$

Таким образом, операция одновременно разбивает изображение на патчи и проецирует их в пространство признаков фиксированной размерности.

В отличие от MLP-Mixer, здесь сохраняется пространственная структура данных: результат остаётся в виде тензора.

Основной блок слоя ConvMixer состоит из двух операций:

1. Первая — **depthwise convolution**. Это свёртка, которая применяется отдельно к каждому каналу. Она отвечает за пространственное смешивание информации. Важно, что здесь лучше работают необычно большой размер ядра (например, 7 или 9) - это позволяет учитывать широкий контекст уже на одном слое. После применяется residual-связь. $z'_l = \text{BN}(\sigma(\text{ConvDepthwise}(z_{l-1}))) + z_{l-1}$
2. Далее следует **pointwise convolution** (1×1), которая смешивает информацию между каналами. Таким образом, как и в MLP-Mixer, здесь явно разделены две операции: пространственная и канальная, но реализованы они через свёртки. $z_{l+1} = \text{BN}(\sigma(\text{ConvPointwise}(z'_l)))$

Каждый слой сопровождается функцией активации (GELU или ReLU) и BatchNorm, что соответствует современным практикам обучения CNN.

Вся модель строится как последовательность таких блоков. После нескольких повторений применяется глобальное усреднение (global pooling), и полученный вектор подаётся на линейный softmax классификатор.

2.3. Пример

Для изображения 224×224 и патча 16×16 получаем карту $14 \times 14 \times C$. Depthwise conv с ядром 7 даёт широкий receptive field. Stacking слоёв формирует глобальный контекст.

Довольно забавен тот факт, что авторы вынесли большую часть экспериментов в приложения, обосновав это тем, что для простой идеи соблюдение стандартных 10 страниц на статью с простой но хорошей идеей избыточно, тем самым основной текст статьи занимает лишь 4,5 страницы. Они даже написали весь код класса предложенной модели буквально в 5 строчек, сказав “что он поместится в один короткий пост”. (Рис. 7)

```
def ConvMixer(h,d,k,p,n):
    S,C,A=Sequential,Conv2d,lamba x:S(x,GELU(),BatchNorm2d(h))
    R=type('',(S,),'forward':lamba s,x:s[0](x)+x)
    return S(A(C(3,h,p,p)),*[S(R(A(C(h,h,k,groups=h,padding=k//2))),A(C(h,h,1))) for
    i in range(d)],AdaptiveAvgPool2d(1),Flatten(),Linear(h,n))
```

Рис. 7: Реализация ConvMixer

2.4. Экспериментальные результаты

В общем, авторы проводят эксперименты как на ImageNet, так и на CIFAR-10. На ImageNet ConvMixer показывает точность порядка 80–81%, что сопоставимо с ResNet и близко к Transformer-моделям при сопоставимом числе параметров. При этом подчёркивается, что из-за ограниченных вычислительных ресурсов авторы не проводили никакой настройки гиперпараметров, то есть модель не была специально оптимизирована под максимальную точность, что делает результат ещё более показательным.

Comparison with other simple models trained on ImageNet-1k only with input size 224.							
Network	Patch Size	Kernel Size	# Params ($\times 10^6$)	Throughput (img/sec)	Act. Fn.	# Epochs	ImNet top-1 (%)
ConvMixer-1536/20★	7	9	51.6	134	G	150	82.20
ConvMixer-1536/20 ●	7	9	51.6	134	G	150	81.37
ConvMixer-1536/20★	7	3	49.4	246	G	150	81.60
ConvMixer-1536/20	7	3	49.4	246	G	150	80.43
ConvMixer-1536/20	14	9	52.3	538	G	150	78.92
ConvMixer-1536/24★	14	9	62.3	447	G	150	80.21
ConvMixer-768/32 ●	7	7	21.1	206	R	300	80.16
ConvMixer-1024/16	7	9	19.4	244	G	100	79.45
ConvMixer-1024/12	7	8	14.6	358	G	90	77.75
ConvMixer-512/16	7	8	5.4	599	G	90	73.76
ConvMixer-512/12 ●	7	8	4.2	798	G	90	72.59
ConvMixer-768/32	14	3	20.2	1235	R	300	74.93
ConvMixer-1024/20 ●	14	9	24.4	750	G	150	76.94
ResNet-152 ●	—	3	60.2	828	R	150	79.64
ResNet-101 ●	—	3	44.6	1187	R	150	78.33
ResNet-50	—	3	25.6	1739	R	150	76.32
DeiT-B†	7	—	86.7	83	G	—	—
DeiT-S†	7	—	22.1	174	G	—	—
DeiT-Ti†	7	—	5.7	336	G	—	—
DeiT-B ●	16	—	86	792	G	300	81.8
DeiT-S ●	16	—	22	1610	G	300	79.8
DeiT-Ti ●	16	—	5.7	2603	G	300	72.2
ResMLP-S12/8 ●	8	—	22.1	872	G	400	79.1
ResMLP-B24/8 ●	8	—	129	181	G	400	81.0
ResMLP-B24	16	—	116	1597	G	400	81.0
Swin-S ●	4	—	50	576	G	300	83.0
Swin-T ●	4	—	29	878	G	300	81.3
ViT-B/16 ●	16	—	86	789	G	300	77.9
Mixer-B/16 ●	16	—	59	1025	G	300	76.44
Isotropic MobileNetv3 ●	8	3	20	—	R	—	80.6
Isotropic MobileNetv3 ●	16	3	20	—	R	—	77.6

Рис. 8: Сравнение моделей

Особенно подробно исследуется влияние различных гиперпараметров (Рис. 8):

- Во-первых, изучается влияние размера ядра depthwise свёртки. Эксперименты показывают, что увеличение размера ядра значительно улучшает качество. Например, при переходе от ядра 3 к 5 точность увеличивается примерно на 1.5%, а дальнейшее увеличение до 7 и 9 даёт дополнительный прирост. Это подтверждает гипотезу о том, что большой receptive field критически важен. Интересно, что попытка заменить большое ядро на несколько маленьких (то есть

увеличить глубину вместо размера ядра) даёт худший результат. Это означает, что большие ядра не просто заменяют глубину, а дают качественно иной эффект.

- Во-вторых, исследуется влияние размера патча. Увеличение размера патча приводит к снижению точности. Это связано с потерей пространственной информации: чем крупнее патчи, тем меньше деталей сохраняется .
- Также рассматривается влияние ширины и глубины сети. Увеличение как глубины, так и ширины приводит к росту качества, но при этом более широкие сети сходятся быстрее, а более глубокие требуют больше времени на обучение .

На CIFAR-10 модель показывает очень высокую точность — более 96% (Рис. 9) при относительно небольшом числе параметров. При этом важную роль играют методы аугментации данных, такие как RandAugment, mixup и cutmix, которые дают существенный прирост качества (до нескольких процентов).

Tiny ConvMixers trained on CIFAR-10.						
Width h	Depth d	Patch Size p	Kernel Size k	# Params ($\times 10^3$)	Weight Decay	CIFAR-10 Acc. (%)
128	4	1	8	103	0	91.26
128	8	1	8	205	0	93.83
128	12	1	8	306	0	94.83
256	4	1	8	338	0	93.37
256	8	1	8	672	0	95.60
256	12	1	8	1006	0	96.39
256	16	1	8	1339	0	96.74
256	20	1	8	1673	0	96.67
↓ Kernel adjustments						
256	8	1	3	559	0	93.61
256	8	1	5	592	0	95.19
256	8	1	7	641	0	95.80
256	8	1	9	707	0	95.88
256	8	1	11	788	0	95.70
256	8	1	13	887	0	96.04
256	8	1	15	1001	0	96.08
↓ Patch adjustments						
256	8	2	9	709	0	95.00
256	8	4	9	718	0	92.61
256	8	8	9	755	0	85.57
↓ Weight decay adjustments						
256	8	1	9	707	1×10^{-1}	95.88
256	8	1	9	707	1×10^{-2}	96.03
256	8	1	9	707	1×10^{-3}	95.76
256	8	1	9	707	1×10^{-4}	95.63
256	8	1	9	707	1×10^{-5}	95.88

Рис. 9: Сравнение ConvMixer при различных гиперпараметрах

Заключение

Рассмотренные архитектуры показывают важный результат – высокая точность достигается не только за счёт сложных механизмов (attention или сложные CNN), но и за счёт правильной организации обработки данных.

Основные выводы:

- MLP-Mixer: требует много данных, но прост
- ConvMixer: использует свёртки вместо MLP, но сохраняет идею патчей

Общий вывод: важнее организация обработки (space/channel separation), чем сложность блоков. Архитектурные «инновации» не обязательно должны быть такими сложными, как кажется.

Список литературы

- [1] Tolstikhin Ilya, Houlsby Neil, Kolesnikov Alexander et al. MLP-Mixer: An all-MLP Architecture for Vision. — 2021. — [2105.01601](#).
- [2] Trockman Asher, Kolter J. Zico. Patches Are All You Need? — 2022. — [2201.09792](#).